

Retrieval-Augmented Generation for Domain-Specific Course Material Question Answering

FAN JIA-CI (314551028), XXX (413551033), XXX (314551001), and XXX (414551002)

Department of Computer Science
National Yang Ming Chiao Tung University

Abstract

This project presents a Retrieval Aggregated Generation (RAG) system designed to answer domain-specific questions based on course requirements and technical assignment documents. By integrating Large Language Models (LLM) with a retrieval mechanism, the system addresses the challenges of hallucination and context window limitations common in standard LLM usage. We compare a baseline approach using fixed-length chunking against an improved approach utilizing semantic chunking and LLM-driven query rewriting. Evaluation on a ground truth dataset shows that the improved methods increased Recall@3 from 0.8000 to 0.9000. We further conduct a detailed error analysis to identify limitations regarding ordinal sensitivity and semantic interference.

1 Introduction

In the educational domain, students often need to extract specific details—such as submission deadlines, group sizes, or scoring metrics—from lengthy PDF courses. Relying solely on general-purpose Large Language Models (LLMs) to answer these questions can be problematic due to potential hallucinations or the inability to fit entire course histories into a context window. Furthermore, direct processing of large documents via commercial LLM APIs can be cost-prohibitive.

This project implements a local Information Retrieval (IR) system to address these issues. We focus on the task of Retrieval-Augmented Generation (RAG), specifically Document QA. The system ingests PDF documents, indexes them into a vector database, and retrieves relevant context to ground the LLM's responses. The primary goal is to build a cost-effective, accurate system that reduces hallucinations by providing evidence-based answers from specific course materials.

2 Related Work

3 Dataset

3.1 Data Source

The dataset consists of technical course documents from the "Generative Information Retrieval" course, specifically chosen to test different retrieval challenges:

Code Retrieval Assignment (HW1) A technical document describing requirements for a code retrieval task. Details the implementation rules for the sparse (TF-IDF, BM25) and Dense Retrieval methods, together with the structure of the data set and the Recall@10 scoring metric.

Multimodal Retrieval Assignment (HW2)

Focuses on Multimodal Information Retrieval tasks. This document describes the requirements for image-text alignment using Dual Encoders or Vision Language Models (VLMs), with performance evaluated via Recall@3 on a Kaggle dataset.

Final Project Guidelines A comprehensive guide defining the scope of the course's final project (covering topics like Agentic IR and Domain-Specific RAG). It acts as the primary source of knowledge for administrative queries, such as group constraints, submission deadlines, and grade rubrics.

OR-ShARC Benchmark A conversational QA dataset designed to test rule-following capabilities. We utilized its development set as a benchmark to compare the semantic understanding of different embedding models (all-MiniLM-L6-v2 vs. all-mpnet-base-v2).

3.2 Data Statistics

We processed a total of **35 pages** of raw PDF documents. Using our semantic chunking strategy, we generated a corpus of **59 chunks** with an average length of **520 characters** (approx. 71 tokens) per chunk. Table 1 summarizes the detailed statistics for each source document.

Document Source	Pages	Chunks	Avg. Tokens
Final Project Req.	8	18	96
HW1 Specifications	13	19	58
HW2 Specifications	14	22	61
Total / Average	35	59	71

Table 1: Statistics of the processed knowledge base.

3.3 Preprocessing

The data preparation pipeline consists of three main stages:

- Text Extraction and Cleaning:** We employed PyMuPDF to extract raw text from PDF documents. To ensure content quality, we implemented a filtering mechanism to remove non-informative elements such as headers, footers, page numbers, and redundant line breaks.
- Semantic Chunking and Vectorization:** Unlike traditional fixed-length splitting, we utilized a **Semantic Chunker** to segment the text based on semantic similarity. Each fragment was then embedded using the sentence transformers model and indexed via **FAISS** for efficient retrieval. The processed data was consolidated into a unified corpus, assigning a globally unique ID to each chunk.
- Ground Truth Annotation:** For evaluation purposes, we constructed a **ground truth dataset**. This set contains manually annotated question-answer pairs, where each question is explicitly linked to the specific chunk IDs from the processed corpus that contain the correct answer.

4 Baseline

4.1 Implementation

Our baseline system utilizes a static sliding window strategy. The core components are defined as follows:

Chunking Strategy We applied a fixed-length splitting method with a window size of 1000 characters and a 200-character overlap to maintain context continuity.

Embedding Model We utilized the lightweight all-MiniLM-L6-v2 model from the sentence-transformers library to generate dense vector representations.

Retrieval Mechanism We implemented a K-Nearest Neighbors (KNN) search using the FAISS library, retrieving the Top-3 most relevant chunks based on cosine similarity.

4.2 Performance

The baseline achieved a **Recall@3 of 0.8000** and an **MRR of 0.5667**. While it successfully answered keyword-heavy questions (e.g., deadlines), it failed on questions requiring broader semantic understanding, such as "total number of students allowed in a group," where the context was diluted by surrounding text in large fixed chunks.

5 Main Approach

5.1 System Architecture

We propose an enhanced RAG framework (Figure 1) that introduces **Semantic Chunking** and **Query Optimization**.

5.2 Methodology

- Semantic Chunking:** Instead of fixed sizes, we use a semantic chunker that splits text based on cosine similarity thresholds between sentences. This ensures that a single chunk contains a coherent topic (e.g., all rules regarding "Grading" stay together).
- Dual-Source Retrieval:** The system retrieves from two sources: the specific Document (assignment file) and a general Knowledge base (supplementary PDFs), allowing the LLM to synthesize background knowledge with specific requirements.
- Query Rewriting:** If the initial retrieval yields low confidence or the LLM detects missing information, the system triggers a "Query Rewrite" loop. The LLM reformulates the user's query into a more search-friendly format to improve vector matching.

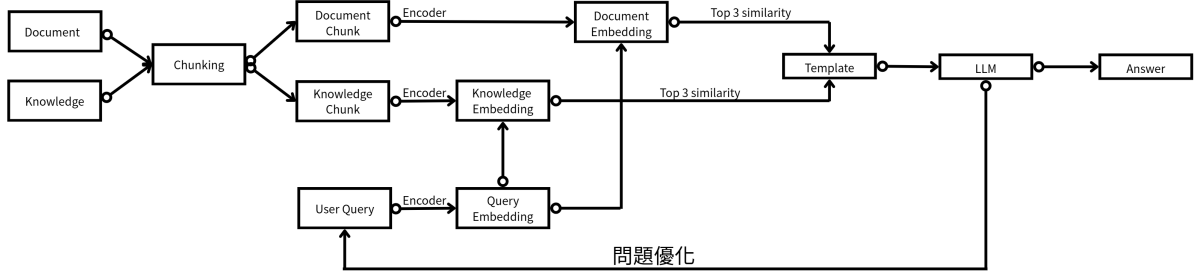


Figure 1: System Architecture: Utilizing Semantic Chunking, Dual-Source Retrieval, and LLM-based Query Rewriting.

4. **Generation:** The top-3 retrieved chunks are fed into the gemini-2.5-flash model to generate the final answer.

6 Evaluation Metric

We evaluated the retrieval component using two standard quantitative metrics:

- **Recall@k:** Measures if the ground-truth chunk is present in the top- k retrieved results.

$$Recall@k = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \mathbb{I}(gt_i \in R_{i,k}) \quad (1)$$

Where gt_i is the ground truth chunk set and $R_{i,k}$ is the set of retrieved chunks.

- **Mean Reciprocal Rank (MRR):** Evaluates the ranking quality, prioritizing correct answers appearing earlier in the list.

7 Results & Analysis

We expanded the test set to 30 questions for the main approach. The implementation of Semantic Chunking significantly improved performance.

Method	Recall@3	MRR
Baseline (Fixed)	0.8000	0.5667
Main (Semantic)	0.9000	0.6222

Table 2: Comparison of retrieval performance.

7.1 Analysis

The increase in Recall@3 to 0.9 indicates that semantic chunking successfully aggregates related information, reducing the "split context" problem. For example, questions about "Mid-term

Project Results" were previously split across arbitrary boundaries but are now contained within single semantic blocks.

Additionally, we compared embedding models using the OR-ShARC dataset. The all-mpnet-base-v2 model achieved a Recall@3 of 0.8326, slightly outperforming the all-MiniLM-L6-v2 (0.8244), suggesting that larger models provide better handling of complex rule-based texts.

8 Error Analysis

Despite the improvements, the system failed on 3 specific queries in the expanded test set. We conducted a qualitative analysis of these failures:

8.1 Case 1: Semantic Interference

Query: "What is the first question that needs to be answered in the report?"

Result: Retrieved chunks about "TA contact info" and "File format".

Analysis: The model was distracted by shared keywords like "question", "submission", and "HW1". The embedding model performed shallow keyword matching rather than understanding the specific intent of finding the *content* of the report questions.

8.2 Case 2: General Term Pollution

Query: "What is the main task in HW2?"

Result: Retrieved chunks about "Title Page" and "Penalties".

Analysis: The terms "HW2" and "Task" are highly generic and appear frequently in headers and administrative sections. The vector space representation of these generic sections overpowered the specific section describing the actual technical task.

8.3 Case 3: Ordinal Insensitivity

Query: "What is the second question... in the report?"

Result: Retrieved the chunk for the *third* question.

Analysis: The embedding model demonstrated an inability to distinguish ordinal directions ("second" vs "third"). Because the text for question 2 and question 3 are semantically similar (both asking about model performance), the model relied on proximity and keyword overlap rather than the specific ordinal constraint.

9 Future Work

Although the current system achieves a high recall, the error analysis reveals specific weaknesses in handling ordinal constraints and generic administrative terms. We propose three targeted optimizations:

- **High-Granularity Embedding Models:** To address the *Ordinal Insensitivity* observed in the error analysis (e.g., distinguishing "second" vs. "third"), we plan to transition from the lightweight all-MiniLM-L6-v2 to deeper models like all-mpnet-base-v2. As shown in our preliminary benchmarks, larger models demonstrate superior capability in capturing fine-grained semantic nuances.
- **Two-Stage Retrieval with Re-ranking:** To mitigate *General Term Pollution*, where generic headers (e.g., "Task Introduction") displace specific content, we propose adding a Cross-Encoder (e.g., BGE-Reranker) after the initial retrieval. This "re-ranking" stage effectively filters out semantically broad administrative text, ensuring that only high-density informational chunks reach the LLM.
- **Proactive Multi-Query Expansion:** While the current system employs a reactive query rewriting mechanism (triggering retries upon failure), we propose a proactive *Query Expansion* strategy. By generating multiple synonymous query variations *before* retrieval, we can broaden the semantic search scope simultaneously, rather than relying on a sequential retry loop.

10 Code

The source code, datasets, and experiment logs are available at:

<https://github.com/megrez33281/>

Retrieval-Augmented-Generation-Document-Retrieval

11 Contribution of Each Member

Student ID	Main Tasks	%
314551028	Preprocessing, FAISS Setup	25%
413551033	RAG Logic, Report Writing	25%
314551001	Experiment Evaluation	25%
414551002	Experiment Evaluation	25%

Table 3: Member contributions and workload proportions.

References

- Y. Sheng, S. Liu, and R. Zhao. 2025. Dynamic chunking and selection for reading comprehension of ultra-long context in large language models. *Proc. 63rd Annual Meeting of the ACL*.
- R. Nogueira and K. Cho. 2019. Passage re-ranking with bert. *arXiv preprint arXiv:1901.04085*.
- N. Muennighoff et al. 2023. Mteb: Massive text embedding benchmark. *NeurIPS Datasets and Benchmarks Track*.